

CSE327 SOFTWARE ENGINEERING

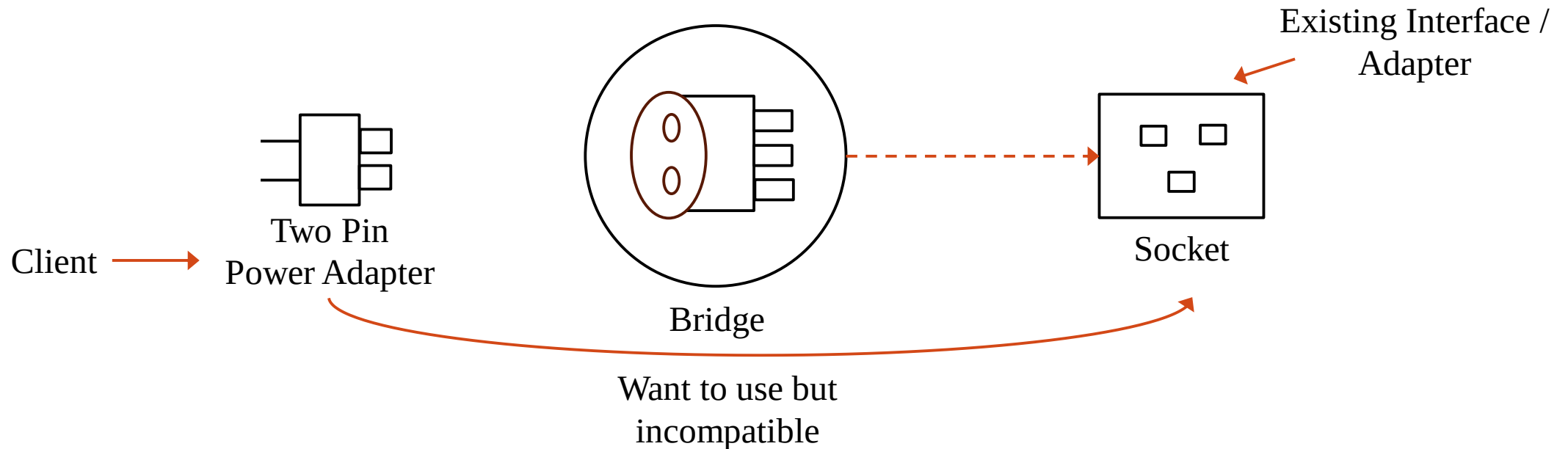


AKM iqtidar Newaz [IqN]

Lecture - 14

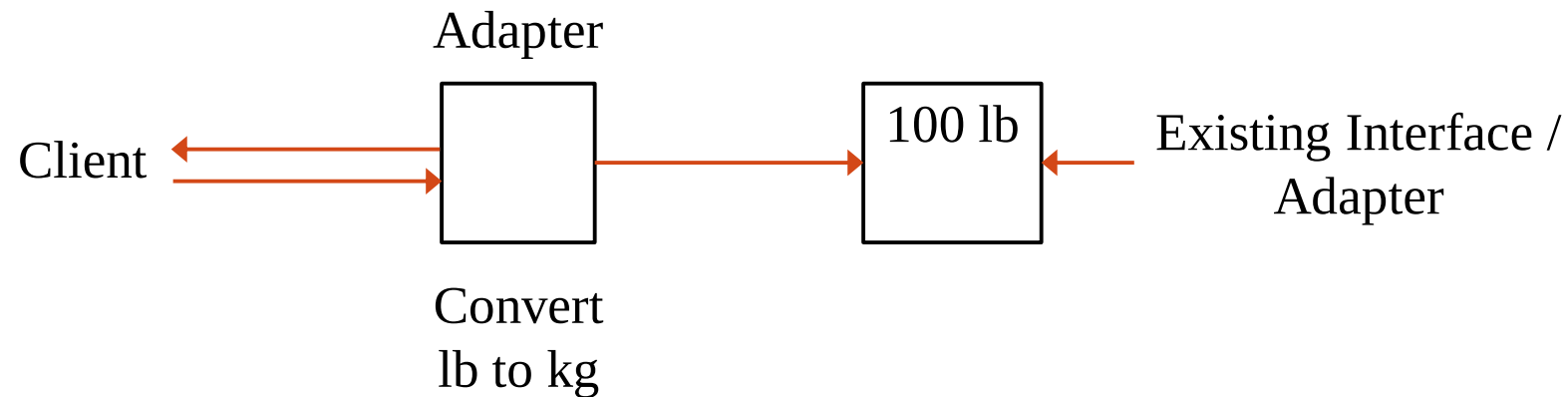
ADAPTER DESIGN PATTERN

Example 1: Power Adapter and Socket



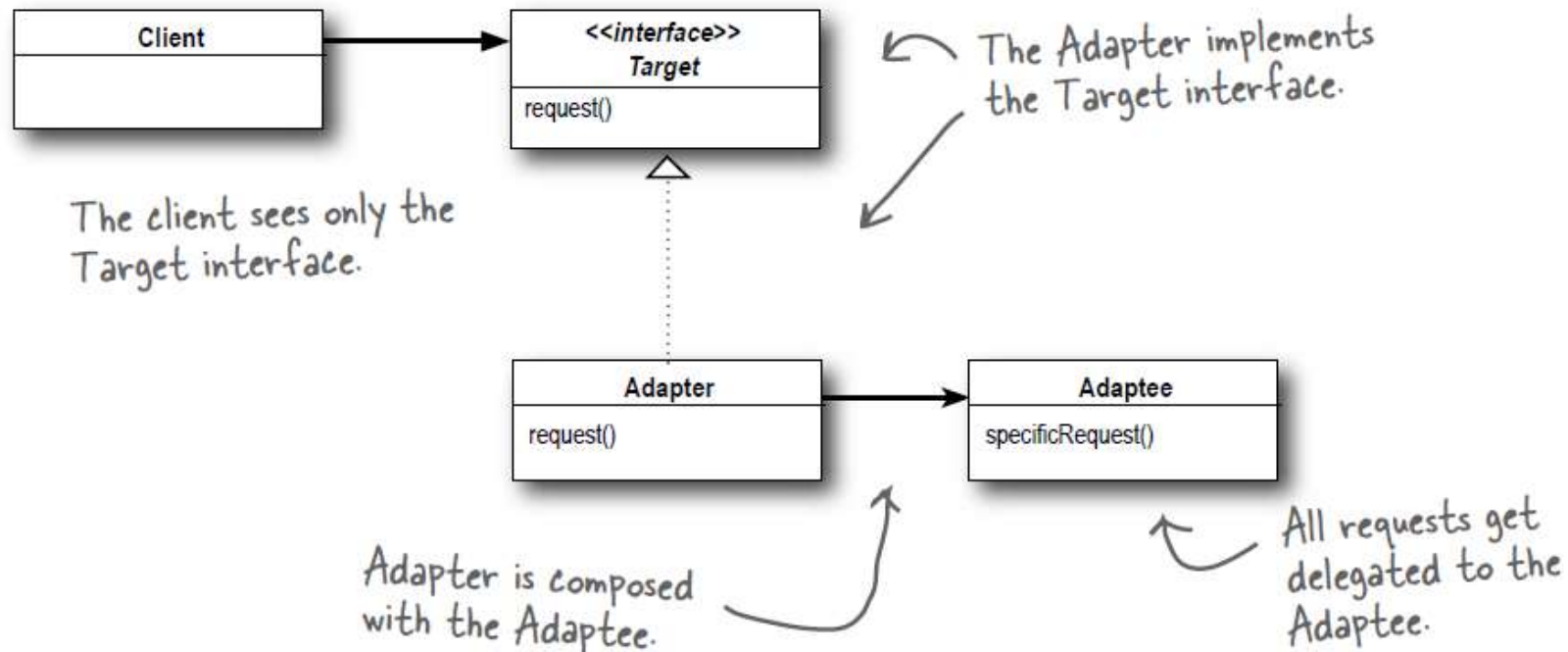
ADAPTER DESIGN PATTERN

Example 2: User wants to see weight in KG:



ADAPTER DESIGN PATTERN (STRUCTURAL DESIGN PATTERN)

- ❑ The Adapter Pattern converts the interface of a class into another interface the clients expect.
- ❑ Adapter lets classes work together that couldn't otherwise because of incompatible interfaces.



ADAPTER DESIGN PATTERN (STRUCTURAL DESIGN PATTERN)

`#pragma once` **duck.h**

```
class Duck
{
public:
    virtual ~Duck() = default;
    virtual void quack() = 0;
    virtual void fly() = 0;
};
```

mallardDuck.h

```
#pragma once
#include "duck.h"
#include <iostream>
using namespace std;

class MallardDuck : public Duck
{
public:
    void quack() override
    {
        cout << "Quack" << endl;
    }
    void fly() override
    {
        cout << "I am flying" << endl;
    }
};
```



ADAPTER DESIGN PATTERN (STRUCTURAL DESIGN PATTERN)

```
#pragma once                turkey.h

class Turkey
{
public:
    virtual ~Turkey() = default;
    virtual void gooble() = 0;
    virtual void fly() = 0;
};
```

```
#pragma once
#include "turkey.h"
#include <iostream>
using namespace std;

wildTurkey.h

class WildTurkey : public Turkey
{
public:
    void gooble() override
    {
        cout << "Googble, gooble" << endl;
    }
    void fly() override
    {
        cout << "I am flying a short distance" << endl;
    }
};
```



ADAPTER DESIGN PATTERN (STRUCTURAL DESIGN PATTERN)

```
#pragma once
#include "turkey.h"
#include "duck.h"
#include <iostream>
using namespace std;

class TurkeyAdapter : public Duck
{
public:
    TurkeyAdapter(Turkey *tur) : turkey(tur) {}

    void quack() override
    {
        turkey->gooble();
    }
    void fly() override
    {
        for (int i = 0; i < 5; i++)
        {
            turkey->fly();
        }
    }

private:
    Turkey *turkey;
};
```

turkeyAdapter.h

```
#pragma once
#include "duck.h"
#include "turkey.h"
#include <iostream>
#include <random>
#include <ctime>

using namespace std;

class DuckAdapter : public Turkey
{
public:
    DuckAdapter(Duck *du) : duck(du), randGen(std::random_device{}()), dist(0, 4) {}

    void gooble() override
    {
        duck->quack();
    }

    void fly() override
    {
        if (dist(randGen) == 0) // Corrected function call
            duck->fly();
    }

private:
    Duck *duck;
    std::mt19937 randGen; // Mersenne Twister RNG
    std::uniform_int_distribution<int> dist; // Uniform distribution
};
```

duckAdapter.h



ADAPTER DESIGN PATTERN (STRUCTURAL DESIGN PATTERN)

```
#include "turkeyAdapter.h"
#include "duckAdapter.h"
#include "mallardDuck.h"
#include "wildTurkey.h"
#include <iostream>
using namespace std;

void testDuck(Duck *duck)
{
    duck->quack();
    duck->fly();
}

void testTurkey(Turkey *turkey)
{
    turkey->gooble();
    turkey->fly();
}
```

main.cpp

```
int main()
{
    Duck *duck = new MallardDuck();
    Turkey *turkey = new WildTurkey();

    cout << "The Duck says..." << endl;
    testDuck(duck);

    cout << "The Turkey says..." << endl;
    testTurkey(turkey);

    cout << "\nTurkey Adapter Performing\n";
    Duck *turkeyAdapter = new TurkeyAdapter(turkey);
    testDuck(turkeyAdapter);

    cout << "\nDuck Adapter Performing\n";
    Turkey *duckAdapter = new DuckAdapter(duck);
    testTurkey(duckAdapter);
}
```

main.cpp

Output:

The Duck says...
Quack
I am flying
The Turkey says...
Gooble, gooble
I am flying a short distance

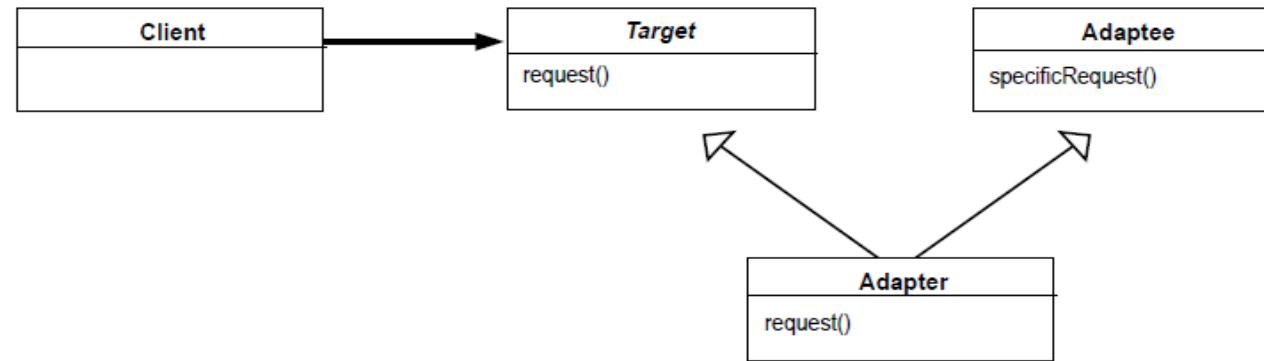
Turkey Adapter Performing
Gooble, gooble
I am flying a short distance
I am flying a short distance
I am flying a short distance
I am flying a short distance
I am flying a short distance

Duck Adapter Performing
Quack
I am flying

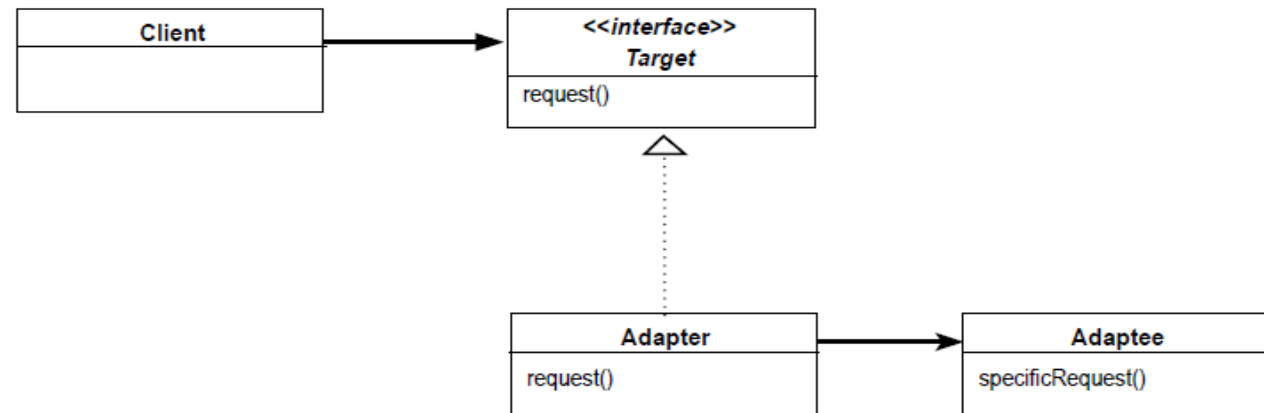


DIFFERENT TYPES OF ADAPTER

Class Adapter



Object Adapter



CLASS ADAPTER: EXAMPLE

duck.h

```
#pragma once

class Duck
{
public:
    virtual ~Duck() = default;
    virtual void quack() = 0;
    virtual void fly() = 0;
};
```

mallardDuck.h

```
#pragma once
#include "duck.h"
#include <iostream>
using namespace std;

class MallardDuck : public Duck
{
public:
    void quack() override
    {
        cout << "Quack" << endl;
    }
    void fly() override
    {
        cout << "I am flying" << endl;
    }
};
```



CLASS ADAPTER: EXAMPLE

```
#pragma once
```

turkey.h

```
class Turkey
{
public:
    virtual ~Turkey() = default;
    virtual void gobble() = 0;
    virtual void fly() = 0;
};
```

```
#pragma once
```

```
#include "turkey.h"
```

```
#include <iostream>
```

```
using namespace std;
```

wildTurkey.h

```
class WildTurkey : public Turkey
```

```
{
```

```
public:
```

```
    void gobble() override
```

```
{
```

```
        cout << "Gobble, Gobble" << endl;
```

```
}
```

```
    void fly() override
```

```
{
```

```
        cout << "I am flying a short distance" << endl;
```

```
}
```

```
};
```



CLASS ADAPTER: EXAMPLE

```
#pragma once
#include "duck.h"
#include "turkey.h"
#include "mallardDuck.h"
#include <random>
#include <ctime>
```

mallardDuckAdapter.h

```
class MallardDuckAdapter : public Turkey, private MallardDuck
{
public:
    MallardDuckAdapter() = default;
    void gobble() override
    {
        MallardDuck::quack();
    }
    void fly() override
    {
        if (dist(randGen) == 0) // Corrected function call
            MallardDuck::fly();
    }

private:
    std::mt19937 randGen; // Mersenne Twister RNG
    std::uniform_int_distribution<int> dist; // Uniform distribution
};
```



CLASS ADAPTER: EXAMPLE

```
#pragma once
#include "duck.h"
#include "wildTurkey.h"
#include <iostream>
using namespace std;
```

wildTurkeyAdapter.h

```
class WildTurkeyAdapter : public Duck, private WildTurkey
{
public:
    WildTurkeyAdapter() = default;
    void quack() override
    {
        WildTurkey::gobble();
    }
    void fly() override
    {
        for (int i = 0; i < 5; i++)
        {
            WildTurkey::fly();
        }
    }
};
```



CLASS ADAPTER: EXAMPLE

```
#include "wildTurkey.h"
#include "wildTurkeyAdapter.h"
#include "mallardDuck.h"
#include "mallardDuckAdapter.h"
#include <iostream>
using namespace std;
```

```
void testDuck(Duck *duck)
{
    duck->quack();
    duck->fly();
}
```

```
void testTurkey(Turkey *turkey)
{
    turkey->gobble();
    turkey->fly();
}
```

```
int main()
{
    Turkey *turkey = new WildTurkey();
    cout << "The Turkey says: " << endl;
    turkey->gobble();
    turkey->fly();

    cout << "\nTurkeyAdapter is coming:\n";
    Duck *wildTurkeyAdapter = new WildTurkeyAdapter();
    testDuck(wildTurkeyAdapter);

    cout << "\nDuckAdapter is coming:\n";
    Turkey *duck = new MallardDuckAdapter();
    testTurkey(duck);
}
```

Output:

The Turkey says:
Gobble, Gobble
I am flying a short distance

TurkeyAdapter is coming:
Gobble, Gobble
I am flying a short distance
I am flying a short distance
I am flying a short distance
I am flying a short distance
I am flying a short distance

DuckAdapter is coming:
Quack

